LEARN BEING FORWARD

Programming — C++ Programming

# Classes, inheritance & polymorphism

COMPLETE LEARNING GUIDE

### What You'll Learn

Classes, inheritance, and polymorphism are the heart of object-oriented C++, and they're also where the language's most consequential and most frequently mis-learned rules live: when a destructor must be virtual, what the Rule of Three actually protects against, and why forgetting override on a derived method can silently produce a completely different (and wrong) function signature instead of a compile error. This sub-module goes deep on class anatomy and constructor behavior, the Rule of Three for classes that manage a resource, public inheritance and access control across a hierarchy, virtual functions and how dynamic dispatch actually works at runtime, abstract base classes, the specific danger of a non-virtual destructor, the diamond problem in multiple inheritance, and operator overloading — each anchored in a compilable example so the underlying mechanism, not just the keyword, sticks.

## 1. Class Anatomy: Access Specifiers, Constructors, and Destructors

A class's members default to private visibility (a struct's default to public — the only structural difference between the two keywords in C++), and public/protected/private sections can appear in any order and any number of times within a class body. A constructor (same name as the class, no return type) runs automatically when an object is created and is responsible for establishing a valid initial state; a destructor (~ClassName()) runs automatically when the object is destroyed (goes out of scope, or is deleted) and is responsible for releasing any resources the object acquired. Both are optional — the compiler generates a default (no-argument) constructor and a default destructor if none is written, but only the default constructor disappears silently the moment you define ANY constructor yourself.

```cpp
#include <iostream>

class Account {
public:
    Account(double initialBalance) : balance_(initialBalance) {
        std::cout << "Account opened with balance " << balance_ << "\n";
    }
    ~Account() {
        std::cout << "Account closed, final balance " << balance_ << "\n";
    }
    void deposit(double amount) { balance_ += amount; }
private:
    double balance_;
};

int main() {
    Account a(100.0);
    a.deposit(50.0);
    return 0; // destructor runs automatically here
}
```

## 2. Constructor Initialization Lists and the Rule of Three

Member initializer lists (ClassName(int x) : member_(x) {}) initialize members directly in the order they're DECLARED in the class (not the order listed in the initializer list — mismatched order only produces a compiler warning, which is easy to miss), and are mandatory for const members, reference members, and base-class constructors that take arguments. The Rule of Three states that if a class defines any one of a custom destructor, copy constructor, or copy-assignment operator, it almost always needs to define all three — because the default compiler-generated versions of the other two will do a shallow, member-by-member copy that's wrong for any class managing a resource like raw heap memory, leading to double-frees or dangling pointers when two objects end up pointing at the same resource.

- If a class manually manages a raw resource (new/delete, a file handle), the compiler-generated copy constructor's default shallow copy is almost always a bug waiting to happen.
- Modern C++ guidance is actually the 'Rule of Zero': avoid manual resource management entirely by using RAII types (std::string, std::vector, smart pointers) as members, so the compiler-generated special members are correct by construction.

## 3. Inheritance: Base and Derived Classes

class Derived : public Base establishes an is-a relationship — a Derived object contains a full Base object as its foundation and can be used anywhere a Base is expected (via a Base pointer or reference). public inheritance (overwhelmingly the most common) preserves each base member's original access level in the derived class; protected inheritance downgrades public base members to protected; private inheritance makes all inherited members private in the derived class. A derived class's constructor must initialize its base class portion, either implicitly (calling Base's default constructor) or explicitly via the initializer list when Base requires arguments.

```cpp
#include <iostream>

class Animal {
public:
    Animal(const std::string &name) : name_(name) {}
    void sleep() const { std::cout << name_ << " is sleeping.\n"; }
protected:
    std::string name_;
};

class Dog : public Animal {
public:
    Dog(const std::string &name) : Animal(name) {} // must initialize the base class
    void bark() const { std::cout << name_ << " says Woof!\n"; } // protected member accessible here
};

int main() {
    Dog d("Rex");
    d.sleep(); // inherited from Animal
    d.bark();  // defined in Dog
    return 0;
}
```

## 4. Virtual Functions and Dynamic Dispatch

Marking a base class member function virtual tells the compiler that calls to it through a base-class pointer or reference should be resolved at RUNTIME based on the object's actual (most-derived) type, not its static (declared) type — this is dynamic dispatch, and it's what makes polymorphism work: a single Animal* could point at a Dog or a Cat, and calling a virtual makeSound() on it invokes whichever override actually matches the real object. The override keyword on a derived class's overriding function is not required by the compiler but should always be used anyway — it makes the compiler verify the function genuinely overrides a matching virtual base function, catching signature mismatches (a wrong parameter type, a missing const) that would otherwise silently create an unrelated new function instead of an override.

- Without virtual, calling a function through a Base* always calls Base's version, even if the object is actually a Derived — this is static dispatch, resolved at compile time.
- Always add override on every intended overriding function — it turns a silent 'accidentally created a new function' bug into a compile error.

```cpp
#include <iostream>

class Shape {
public:
    virtual double area() const { return 0.0; }
    virtual ~Shape() = default; // virtual destructor — see next section
};

class Circle : public Shape {
public:
    Circle(double r) : radius_(r) {}
    double area() const override { return 3.14159 * radius_ * radius_; }
private:
    double radius_;
};

int main() {
    Shape *s = new Circle(2.0);
    std::cout << s->area() << "\n"; // 12.566... — dynamic dispatch picks Circle::area
    delete s;
    return 0;
}
```

## 5. Abstract Classes and Pure Virtual Functions

A pure virtual function (virtual double area() const = 0;) has no implementation in its base class and forces every concrete derived class to provide one — a class containing at least one pure virtual function becomes an abstract class that cannot be instantiated directly, only used as a base for derived classes that implement all of its pure virtual functions. Abstract classes are how C++ expresses an interface-like contract: 'every Shape must know how to compute its own area, but Shape itself doesn't know how, because it depends entirely on which specific shape it is.'

```cpp
class Shape {
public:
    virtual double area() const = 0; // pure virtual — no implementation, no body
    virtual ~Shape() = default;
};

// Shape s; // COMPILE ERROR — cannot instantiate an abstract class

class Square : public Shape {
public:
    Square(double side) : side_(side) {}
    double area() const override { return side_ * side_; } // required implementation
private:
    double side_;
};
```

## 6. Virtual Destructors and Polymorphic Deletion

If a base class will ever be deleted through a base-class pointer that might actually point to a derived object (exactly the Shape *s = new Circle(...); delete s; pattern above), the base class's destructor MUST be virtual — otherwise delete s only runs Shape's destructor, never Circle's, silently skipping any cleanup Circle's destructor was responsible for and producing a resource leak or worse. This single rule — 'if a class has any virtual function, give it a virtual destructor too' — is one of the most important and most frequently violated rules in C++ class design, and most style guides and static analyzers flag a non-virtual destructor in a class with virtual functions as an error-level issue, not a mere suggestion.

- Rule of thumb: a base class intended for polymorphic use should ALWAYS declare virtual ~ClassName() = default;, even if it does nothing else special.
- Deleting a derived object through a non-virtual base-class destructor is undefined behavior — it typically runs only the base destructor, leaking any derived-only resources.

## 7. Multiple Inheritance and the Diamond Problem

C++ allows a class to inherit from more than one base class simultaneously, which creates the classic 'diamond problem' when two base classes both inherit from a common ancestor: a derived class inheriting from both ends up with two separate copies of that common ancestor's data unless virtual inheritance (class B : virtual public A) is used to ensure only one shared copy exists. Multiple inheritance is powerful but genuinely tricky to reason about correctly, and in modern C++ it is generally reserved for mixing in small, focused interface-like abstract classes rather than combining multiple classes that each carry their own real state.

```cpp
class Animal { public: std::string name; };
class Swimmer : virtual public Animal {};  // virtual inheritance
class Flyer   : virtual public Animal {};   // shares ONE Animal, not two
class Duck    : public Swimmer, public Flyer {}; // diamond — resolved by virtual inheritance
```

## 8. Operator Overloading

C++ lets you define what the built-in operators mean for your own class types — overloading operator+ to add two Vector2D objects component-wise, for instance — which lets custom types participate in natural, readable expressions instead of requiring named methods like .add(other) for every operation. Overloaded operators should behave the way users intuitively expect from the built-in types (operator== should actually test equality, operator+ shouldn't have side effects), and are typically implemented as either member functions (when the left operand is always the class itself) or free functions (needed when the left operand might be a different type, like a plain number).

```cpp
#include <iostream>

class Vector2D {
public:
    Vector2D(double x, double y) : x_(x), y_(y) {}
    Vector2D operator+(const Vector2D &other) const {
        return Vector2D(x_ + other.x_, y_ + other.y_);
    }
    void print() const { std::cout << "(" << x_ << ", " << y_ << ")\n"; }
private:
    double x_, y_;
};

int main() {
    Vector2D a(1, 2), b(3, 4);
    Vector2D c = a + b; // calls operator+
    c.print();          // (4, 6)
    return 0;
}
```

## Common Pitfalls

- Giving a base class virtual functions but a non-virtual destructor, causing derived-class cleanup to be silently skipped on polymorphic deletion.
- Omitting the override keyword, so a signature mismatch creates an unrelated new function instead of a compile-time error.
- Relying on the compiler-generated copy constructor/assignment for a class that manually manages a raw resource, causing shallow-copy double-frees.
- Passing or returning a derived object by value through a base-class-typed parameter/variable, causing object slicing that discards the derived portion.
- Instantiating a class containing pure virtual functions by forgetting one of them is still unimplemented in a 'concrete' derived class.
- Using multiple inheritance without virtual inheritance when two bases share a common ancestor, duplicating that ancestor's state unexpectedly.

## Key Takeaways

- A base class used polymorphically must have a virtual destructor, or derived-class cleanup is silently skipped.
- Always write override on intended overriding functions — it converts silent signature-mismatch bugs into compile errors.
- The Rule of Three (destructor, copy constructor, copy assignment) applies to any class manually managing a resource; the Rule of Zero (avoid manual management via RAII members) is the preferred modern alternative.

- Pure virtual functions (= 0) make a class abstract, uninstantiable until every pure virtual function is implemented by a derived class.
- Virtual inheritance resolves the diamond problem by ensuring only one shared copy of a common base exists.
- Operator overloading should match the operator's built-in intuitive meaning, not repurpose it for unrelated behavior.

## Learn More

**cppreference — Classes**
https://en.cppreference.com/w/cpp/language/classes

**cppreference — Virtual function specifier**
https://en.cppreference.com/w/cpp/language/virtual

**isocpp.org — C++ FAQ**
https://isocpp.org/faq